Why this SSL error:

In simple terms with analogy:

How flow happens:

Normal: You send a locked letter to Gemini using a delivery person with a badge you trust (certificate).

With Zscaler: A security guard opens your letter, checks it, then sends it in a new box with their badge. Python says, "I don't know this badge!"

Fix: Give Python the guard's badge so it knows the guard is trusted and lets the letter pass.

---

the whole point of this step is security.

Right now, Python is refusing HTTPS connections because it doesn't trust the Zscaler-issued certificate. That's good in the sense that Python is protecting you from "**man-in-the-middle**" attacks.

The reason you need to add Zscaler's certificate is:

In corporate network, Zscaler intercepts HTTPS traffic to scan for threats.

To do that, it replaces the website's real certificate with one signed by Zscaler's own CA.

Python doesn't trust that CA by default — so it blocks the connection.

By adding only your company's Zscaler CA cert to Python's trusted store, you're telling Python:

> "Yes, I know Zscaler is intercepting traffic — this is expected and safe in this network."

---

so, during ssl handshake , this is happening :

Your request still reaches Gemini, and Gemini's response comes back.

But on the way back:

1. Zscaler catches the reply.

2. It decrypts it to scan for threats.

3. Then it re-encrypts it with Zscaler's own certificate.

4. Python checks the cert and says:

> “This isn’t Gemini’s certificate! I don’t know this signer.”

That’s why you need to give Python a copy of Zscaler’s certificate — so it says:

> “Ah, this badge belongs to our security guard — I trust them.”

My cmd trace:

C:\Users\harsh_vyas1>**python -m venv sslfixenv**

C:\Users\harsh_vyas1>**sslfixenv\Scripts\activate**

(sslfixenv) C:\Users\harsh_vyas1>**python cert.py**

Traceback (most recent call last):

File "C:\Users\harsh_vyas1\cert.py", line 31, in <module>

import requests as r

ModuleNotFoundError: No module named 'requests'

(sslfixenv) C:\Users\harsh_vyas1>**pip install requests**

Collecting requests

Using cached requests-2.32.4-py3-none-any.whl.metadata (4.9 kB)

Collecting charset_normalizer<4,>=2 (from requests)

Using cached charset_normalizer-3.4.3-cp313-cp313-win_amd64.whl.metadata (37 kB)

Collecting idna<4,>=2.5 (from requests)

Using cached idna-3.10-py3-none-any.whl.metadata (10 kB)

Collecting urllib3<3,>=1.21.1 (from requests)

Using cached urllib3-2.5.0-py3-none-any.whl.metadata (6.5 kB)

Collecting certifi>=2017.4.17 (from requests)

Using cached certifi-2025.8.3-py3-none-any.whl.metadata (2.4 kB)

Using cached requests-2.32.4-py3-none-any.whl (64 kB)

Using cached certifi-2025.8.3-py3-none-any.whl (161 kB)

Using cached charset_normalizer-3.4.3-cp313-cp313-win_amd64.whl (107 kB)

Using cached idna-3.10-py3-none-any.whl (70 kB)

Using cached urllib3-2.5.0-py3-none-any.whl (129 kB)

Installing collected packages: urllib3, idna, charset_normalizer, certifi, requests

Successfully installed certifi-2025.8.3 charset_normalizer-3.4.3 idna-3.10 requests-2.32.4 urllib3-2.5.0

[notice] A new release of pip is available: 24.3.1 -> 25.2

[notice] To update, run: **python.exe -m pip install --upgrade pip**

(sslfixenv) C:\Users\harsh_vyas1>**python cert.py**

Adding Cert

(sslfixenv) C:\Users\harsh_vyas1> deactivate

Fixing SSL Certificate Issues Using cert.py Script

This process fixes SSL certificate errors by installing the correct certificate using an automated Python script.

Solution 1 :

Steps to Execute:

1. Get the Script

Go to your project's Git repository : GitHub - refrepos/AI_Labs at nixon-kurian-patch-1

Download the file cert.py or copy its code to your local machine.

---

2. Create a Virtual Environment

A virtual environment keeps dependencies isolated.

Windows Command Prompt:

python -m venv sslfixenv

---

3. Activate the Virtual Environment

Windows Command Prompt:

sslfixenv\Scripts\activate

You should now see (sslfixenv) at the beginning of your terminal line.

---

4. Install Required Dependencies

Inside the activated environment, install the requests library:

pip install requests

---

5. Run the Script

Execute the script to add the certificate:

python cert.py

If successful, you will see a message like:

Adding cert

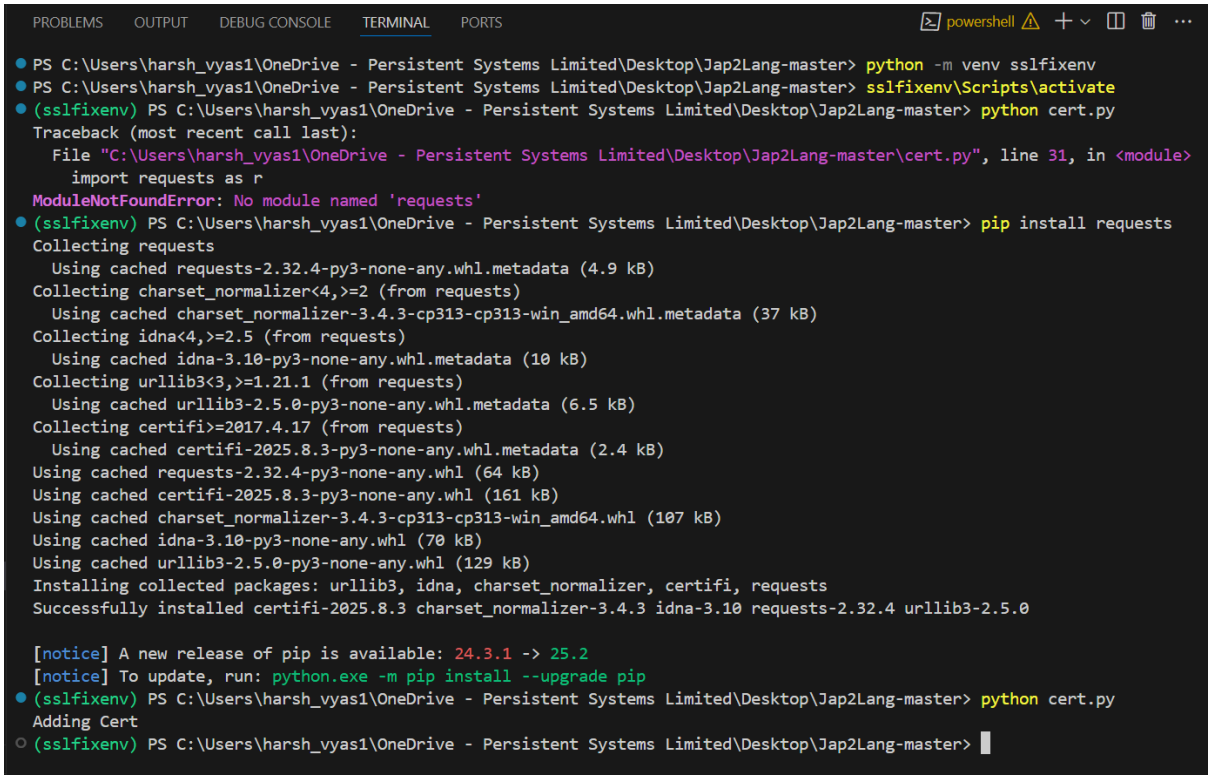
Run your required application in that virtual enviornment

7. Deactivate the Virtual Environment

Once done, you can deactivate the virtual environment:

deactivate

For your project you can set it permanently :



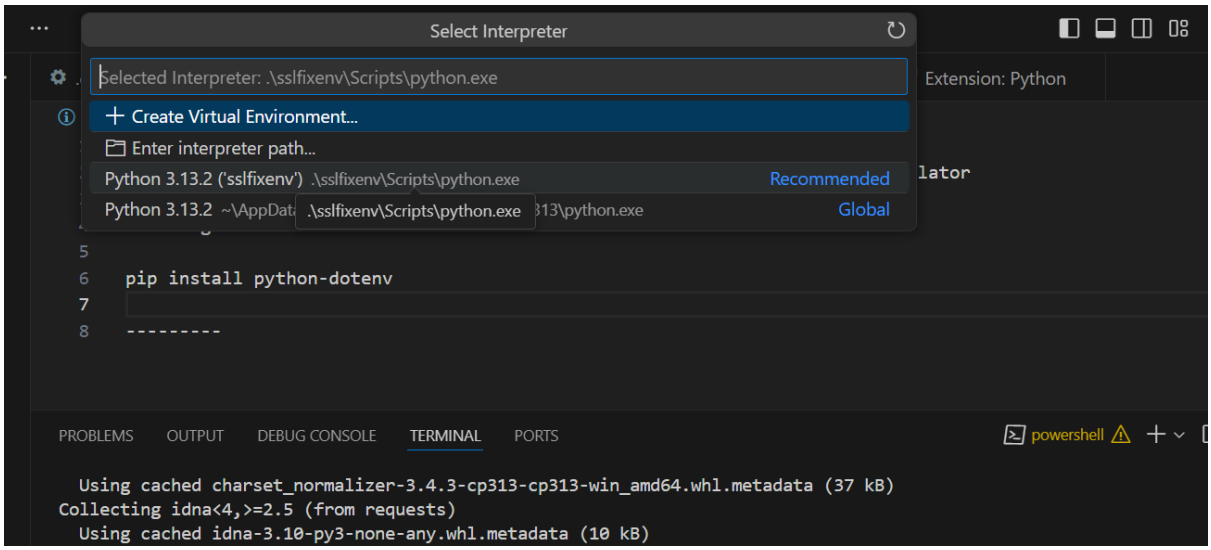
ctrl+shift+P

type :

Python: Select Interpreter

you will see something like: **.\sslfixenv\Scripts\python.exe**

click on it



Now for this project you will not need to add this cert again and also no need to add virtual environment.

Reference : Resolving SSL Certificate Issues with Zscaler: A Complete Guide - Community ssl error

---

cert.py :

SSL_CERT="""-----BEGIN CERTIFICATE-----

MIIE0zCCA7ugAwIBAgIJANu+mC2Jt3uTMA0GCSqGSIb3DQEBCwUAMIGhMQswCQYD

VQQGEwJVUzETMBEGA1UECBMKQ2FsaWZvcm5pYTERMA8GA1UEBxMIU2FuIEpvc2Ux

FTATBgNVBAoTDFpzY2FsZXIgSW5jLjEVMBMGA1UECxMMWnNjYWxlciBJbmMuMRgw

FgYDVQQDEw9ac2NhbGVyIFJvb3QgQ0ExIjAgBgkqhkiG9w0BCQEWE3N1cHBvcnRA

enNjYWxlci5jb20wHhcNMTQxMjE5MDAyNzU1WhcNNDIwNTA2MDAyNzU1WjCBoTEL

MAkGA1UEBhMCVVMxEzARBgNVBAgTCkNhbGlmb3JuaWExETAPBgNVBAcTCFNhbiBK

b3NlMRUwEwYDVQQKEwxac2NhbGVyIEluYy4xFTATBgNVBAsTDFpzY2FsZXIgSW5j

LjEYMBYGA1UEAxMPWnNjYWxlciBSb290IENBMSIwIAYJKoZIhvcNAQkBFhNzdXBw

b3J0QHpzY2FsZXIuY29tMIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEA

qT7STSxZRTgEFFf6doHajSc1vk5jmzmM6BWuOo044EsaTc9eVEV/HjH/1DWzZtcr

fTj+ni205apMTlKBW3UYR+lyLHQ9FoZiDXYXK8poKSV5+Tm0Vls/5Kb8mkhVVqv7

LgYEmvEY7HPY+i1nEGZCa46ZXCOohJ0mBEtB9JVlpDIO+nN0hUMAYYdZ1KZWCMNf

5J/aTZiShsorN2A38iSOhdd+mcRM4iNL3gsLu99XhKnRqKoHeH83lVdfu1XBeoQz

z5V6gA3kbRvhDwoIlTBeMa5l4yRdJAfdpkbFzqiwSgNdhbxTHnYYorDzKfr2rEFM

dsMU0DHdeAZf711+1CunuQIDAQABo4IBCjCCAQYwHQYDVR0OBBYEFLm33UrNww4M

hp1d3+wcBGnFTpjfMIHWBgNVHSMEgc4wgcuAFLm33UrNww4Mhp1d3+wcBGnFTpjf

oYGnpIGkMIGhMQswCQYDVQQGEwJVUzETMBEGA1UECBMKQ2FsaWZvcm5pYTERMA8G

```
A1UEBxMIU2FuIEpvc2UxFTATBgNVBAoTDFpzY2FsZXIgSW5jLjEVMBMGA1UECxMM
WnNjYWxlciBJbmMuMRgwFgYDVQQDEw9ac2NhbGVyIFJvb3QgQ0ExIjAgBgkqhkiG
9w0BCQEWE3N1cHBvcnRAenNjYWxlci5jb22CCQDbvpgtibd7kzAMBgNVHRMEBTAD
AQH/MA0GCSqGSIb3DQEBCwUAA4IBAQAw0NdJh8w3NsJu4KHuVZUrmZgIohnTm0j+
RTmYQ9IKA/pvxAcA6K1i/LO+Bt+tCX+C0yxqB8qzuo+4vAzoY5JEBhyhBhf1uK+P
/WVWFZN/+hTgpSbZgzUEnWQG2gOVd24msex+0Sr7hyr9vn6OueH+jj+vCMiAm5+u
kd7lLvJsBu3AO3jGWVLyPkS3i6Gf+rwAp1OsRrv3WnbkYcFf9xjuaf4z0hRCrLN2
xFNjavxrHmsH8jPHVvgc1VD0Opja0l/BRVauTrUaoW6tE+wFG5rEcPGS80jjHK4S
pB5iDj2mUZH1T8lzYtuZy0ZPirxmtsk3135+CKNa2OCAhhFjE0xd
-----END CERTIFICATE-----"""

import requests as r
cert_path = r.certs.where()

cert_exists = False

with open(cert_path,"r") as f:
  cert_str = f.read()
  if SSL_CERT in cert_str:
    print("Cert Exists")
    cert_exists = True

if cert_exists == False:
  with open(cert_path,"a") as f:
    print("Adding Cert")
    f.write(SSL_CERT)
```

Solution 2: Manually Adding the Zscaler Certificate to Python's Trusted Store

Purpose:

If Python scripts fail due to SSL errors (common with corporate proxies like Zscaler), we can export the Zscaler Root CA from your browser and add it to Python's cacert.pem.

Step 1 — Export Zscaler Certificate from Your Browser

1. Open your browser (Edge/Chrome — steps are similar).

2. Go to Settings.

3. From the sidebar, select:

    In Edge: Privacy, search, and services

    In Chrome: Privacy and security → Security

4. Scroll down to the Security section.

5. Click Manage Certificates (this opens Windows Certificate Manager).

6. Go to the Trusted Root Certification Authorities tab.

7. Scroll through the list until you find Zscaler Root CA.

There might be more than one — either works.

8. Select it and click Export.

9. In the export wizard:

Format: Base-64 encoded X.509 (.CER)

Save it somewhere easy to find (e.g., Downloads/ZscalerRootCA.cer).

Step 2 — Locate Python's Certificate Bundle

1. Open Command Prompt (or terminal).

2. Run:

python -m certifi

3. This prints the file path to Python's certificate bundle — usually something like:

C:\Users\harsh_vyas1\AppData\Local\Programs\Python\Python313\Lib\site-packages\certifi\cacert.pem

4. Copy this file somewhere as a backup before editing (e.g., cacert_backup.pem).

Step 3 — Add Zscaler Certificate to Python's Store

1. Open cacert.pem (from the path above) in a text editor (Notepad works).

2. Open the ZscalerRootCA.cer file you exported earlier in a text editor as well.

You should see something like:

-----BEGIN CERTIFICATE-----

(lots of random characters)

-----END CERTIFICATE-----

3. Copy the entire contents (including the BEGIN and END lines) from the **.cer** file.

4. Paste it at the end of the **cacert.pem** file.

5. Save the file.

Step 4 — Test

Run your Python script again.

If it still fails, double-check:

The cert is correctly pasted with the BEGIN/END markers intact.

No extra spaces or missing lines.

Note: If you work in multiple environments (virtualenv, Conda), you'll need to repeat this process for each environment's cacert.pem.

Warning: Never delete existing certs from cacert.pem. Just append the new one.